

ME308 Industrial Control Course

Lab #6 Post-Lab Report

Arda Barış Tonbil

00031130

Introduction

In sixth lab session of ME308, participants were introduced to RTOS programming via TwinCAT 2 for Beckhoff Industrial PCs. In the task, participants were asked to develop a project from scratch to turn a motor by buttons and monitor it using a signal lamp and a Human-Machine Interface (HMI). In the task, a lock button is for the signal lamp to be allowed to turn on when only one of the buttons are pushed. If motor turns safely with only one of the buttons, then the direction of motor should be observed from the GUI. On the other hand, if both buttons are pushed, an error message should be printed to state that only one of the buttons could be pushed at the same time, since the motor cannot turn CW and CCW at the same time.

Methodology

- To create a project, TwinCAT System Manager should be used in configuration mode. Target should be selected for the software to know the machine model. Then, an I/O device object should be created in the software to interact with the hardware to define I/O connections. In this lab; S3 is used for lock button, S4 and S5 are used for push buttons, H1 is used for signal lamp and K1 and K2 are used for relays of running the motor and selecting its direction. With this configuration file activated, the software can know about the connections relevant to its purpose.
- For this lab, Structural Text Language (STL or ST) is used to write a program with HMI.
- In the code, linear programming is used since logic is relatively simple. There were 3 digital input and 3 digital output variables defined as global variables, while 9 other variables were defined in MAIN program organization unit (POU). Variables were named by the constraint of comprehensibility. Buttons, relays, and signal lamp were declared as global variables, in contrast, variables inside MAIN unit were used for transition purposes.
- In global variable names, A01 refers to the PC itself. Afterwards, each variable has their own code (S3, S4, S5, H1, K1, K2 depending on the component). Then, their functions are verbally written. If they are inputs, they should be also defined as Normally Open (NO) or Normally Closed (NC). Finally, DI refers to Digital Input and DO refers to Digital Output. Their locations on the terminals are also defined as I or Q depending on their types. Finally, types of variables are defined as BOOL for Boolean.

```

VAR_GLOBAL
(*DIGITAL INPUTS*)
  A01_S03_LOCK_BUTTON_NO_DI AT %I* : BOOL;
  A01_S04_PUSH_BUTTON_1_NO_DI AT %I* : BOOL;
  A01_S05_PUSH_BUTTON_2_NO_DI AT %I* : BOOL;
(*DIGITAL OUTPUTS*)
  A01_H01_SIGNAL_LAMP_DO AT %Q* : BOOL;
  A01_K01_MOTOR_RUN_COIL_DO AT %Q* : BOOL;
  A01_K02_MOTOR_DIRECTION_COIL_DO AT %Q* : BOOL;
END_VAR

```

Figure 1: Global variables.

```

VAR
  S3_turned_on:BOOL;
  S4_pushed:BOOL;
  S5_pushed:BOOL;
  motor_runCW:BOOL;
  motor_runCCW:BOOL;
  displayHMI:STRING;
  errorHMI:STRING;
  displayColorHMI:BOOL;
  errorColorHMI:BOOL;
END_VAR

```

Figure 2: Variables defined inside MAIN POU.

```

errorHMI:='NO ERRORS';
errorColorHMI:=FALSE;
IF A01_S03_LOCK_BUTTON_NO_DI THEN
  S3_turned_on:=TRUE;
ELSE
  S3_turned_on:=FALSE;
END_IF

IF A01_S04_PUSH_BUTTON_1_NO_DI THEN
  S4_pushed:=TRUE;
ELSE
  S4_pushed:=FALSE;
END_IF

IF A01_S05_PUSH_BUTTON_2_NO_DI THEN
  S5_pushed:=TRUE;
ELSE
  S5_pushed:=FALSE;
END_IF

IF S4_pushed AND NOT S5_pushed THEN
  motor_runCW:=TRUE;
  motor_runCCW:=FALSE;
END_IF

```

Figure 3: First part of the code.

```

IF S5_pushed AND NOT S4_pushed THEN
    motor_runCCW:=TRUE;
    motor_runCW:=FALSE;
END_IF

IF S4_pushed AND S5_pushed THEN
    motor_runCW:=FALSE;
    motor_runCCW:=FALSE;
    errorHMI:='ONE BUTTON ONLY';
    errorColorHMI:=TRUE;
END_IF

IF NOT S4_pushed AND NOT S5_pushed THEN
    motor_runCW:=FALSE;
    motor_runCCW:=FALSE;
END_IF

IF motor_runCW THEN
    A01_K01_MOTOR_RUN_COIL_DO:=TRUE;
    A01_K02_MOTOR_DIRECTION_COIL_DO:=FALSE;
    displayHMI:='CW';
    displayColorHMI:=TRUE;
    IF S3_turned_on THEN
        A01_H01_SIGNAL_LAMP_DO:=TRUE;
    ELSE
        A01_H01_SIGNAL_LAMP_DO:=FALSE;
    END_IF
END_IF

```

Figure 4: Second part of the code.

```

IF motor_runCCW THEN
    A01_K01_MOTOR_RUN_COIL_DO:=TRUE;
    A01_K02_MOTOR_DIRECTION_COIL_DO:=TRUE;
    displayHMI:='CCW';
    displayColorHMI:=TRUE;
    IF S3_turned_on THEN
        A01_H01_SIGNAL_LAMP_DO:=TRUE;
    ELSE
        A01_H01_SIGNAL_LAMP_DO:=FALSE;
    END_IF
END_IF

IF NOT motor_runCCW AND NOT motor_runCW THEN
    displayHMI:='CW OR CCW?';
    displayColorHMI:=FALSE;
    A01_K01_MOTOR_RUN_COIL_DO:=FALSE;
    A01_K02_MOTOR_DIRECTION_COIL_DO:=FALSE;
    A01_H01_SIGNAL_LAMP_DO:=FALSE;
END_IF

```

Figure 5: Final part of the code.

- When neither S4 or S5 is pushed, HMI prompts the user by asking “CW or CCW?” in Figure 5. Also, no output components are turned on. Also, in Figure 3, error message is initialized as “NO ERRORS”. These components have their color variables initialized as FALSE at the same locations, which will be useful in HMI.
- To check if the buttons are on or off, IF-ELSE statements shown in Figure 3 are used. States of the buttons are stored in variables named S3_turned_on, S4_pushed, and S5_pushed. To run the motor, only one of S4 or S5 can be pushed at the same time. This is checked in Figure 4, and if both buttons are pushed simultaneously, an error message is displayed to warn the operator that both buttons cannot be pushed at the same time. Also, color variable is set to TRUE for error message prompt. As expected, if both buttons are off, motor cannot run, and this is also checked in Figure 4. To turn on the relays, motor_runCW and motor_runCCW variables are set to TRUE when only one of the buttons is pushed in Figures 3 and 4.
- In Figures 4 and 5, depending on motor_runCW and motor_runCCW, relays are turned on. K1 is turned on for both cases to enable the motor, while K2 is enabled for CCW motion only. Similar to previous IF-ELSE statements, FALSE value is explicitly assigned to K2. This practice is essential because the values from previous cycles of execution can be protected to lead to unwanted results. While the motor is moving, color variable for motion prompt is enabled, and if S3 is turned on then signal lamp is enabled. These are checked at the same IF-ELSE statements with motor relays.
- In HMI, two rectangles are used. Left rectangle is used to monitor the motor to see if it is running and which direction it is running in, while right rectangle is used for error prompts. Color of left rectangle is controlled by MAIN.displayColorHMI and color of right rectangle is controlled by MAIN.errorColorHMI. If these variables are on, their respective alarm colors will be displayed instead of plain white.

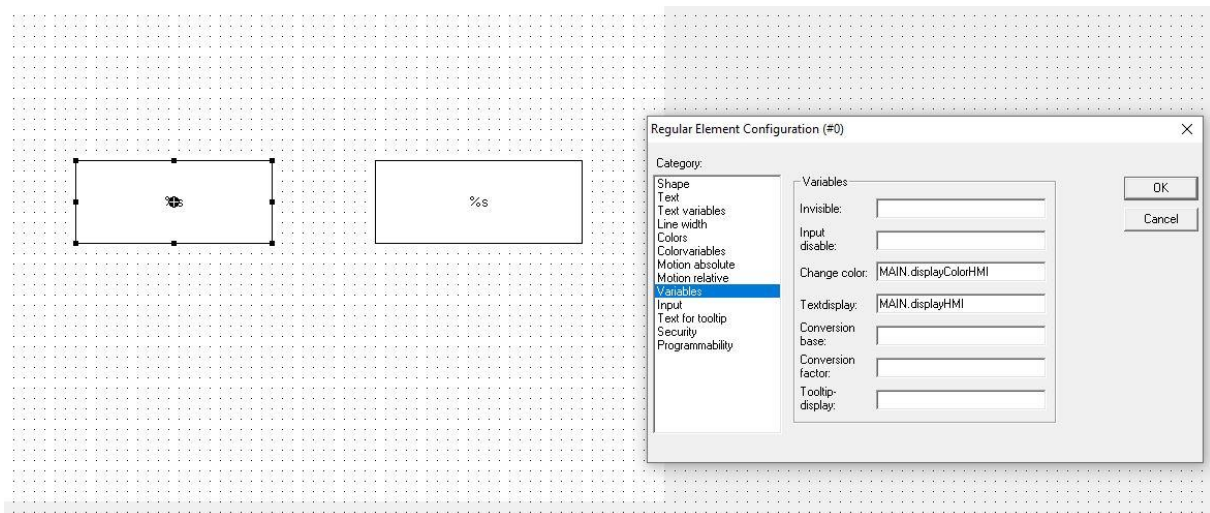


Figure 6: Variable bindings for left rectangle to monitor the motor.

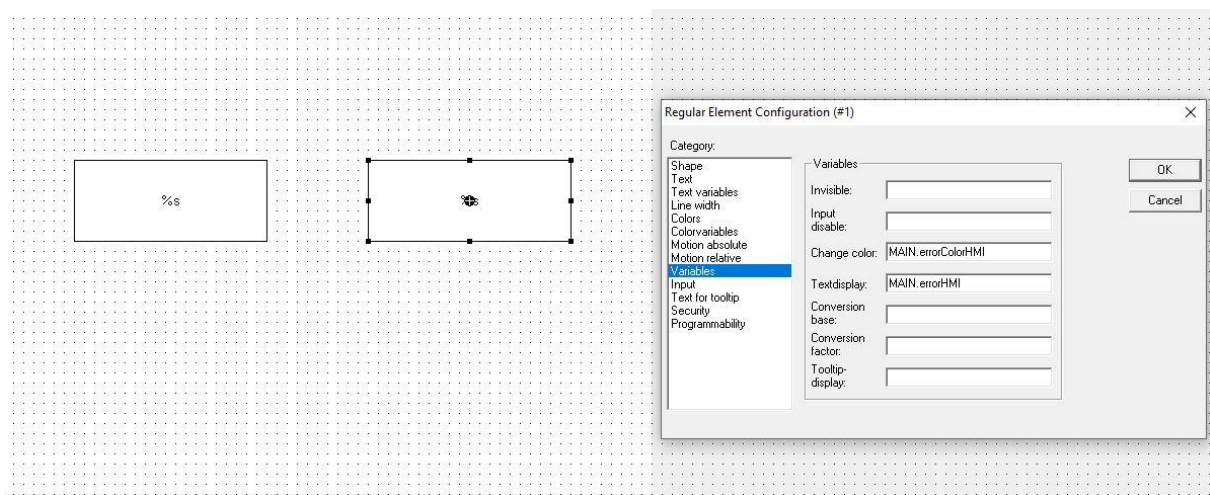


Figure 7: Variable bindings for right rectangle for error prompts.



Figure 8: HMI when neither S4 nor S5 is pushed.



Figure 9: HMI when S4 is pushed and S5 is not pushed (CCW is prompted instead of CW when it's opposite).



Figure 10: HMI when both S4 and S5 are pushed.

Discussion

Sixth lab session of ME308 was an important milestone, as it was the first laboratory session focused on RTOS programming, which provided valuable hands-on experience with Beckhoff Industrial PCs. Beyond the hardware setup, the implementation successfully demonstrated the critical importance of software interlocks in industrial control. The main safety check was the logic designed to handle simultaneous inputs; by detecting if both push buttons were pressed, the system prevented conflicting motor signals and triggered an immediate error state. Consequently, motor was protected from a conflict between two signals.

Another important outcome of this lab session is relevant to necessity of deterministic programming in an RTOS environment. Manually assigning FALSE values to variables was not the main practice in PLC programming, unless they were set or reset manually depending on the application. However, since the values can be latched from previous execution cycles of the code in STL, they should be explicitly reset to FALSE. Finally, participants also had the opportunity to develop and work with an integrated HMI, which provided real-time feedback depending on motor and buttons.